

Vector Databases: A Technical Overview

1. Introduction

A vector database is a specialized storage system designed to index and query high-dimensional numerical vectors, commonly called embeddings. These embeddings are produced by machine learning models that convert unstructured data such as text, images, or audio into a fixed-length array of floating point numbers. Vector databases exist because traditional relational databases, which are optimized for exact matches and range queries on structured columns, perform poorly when the task is to find the "most similar" items in a dataset rather than an exact match. This document explains what vector databases do, how they work internally, the main indexing algorithms in use today, and the trade-offs between the most popular implementations.

2. Why Similarity Search Is Different

In a relational database, a query like "find all orders placed after March 1st" can be answered efficiently using a B-tree index, because the underlying data has a natural ordering. Embeddings do not have a natural ordering. Instead, similarity between two vectors is typically measured using one of three distance or similarity functions:

- Cosine similarity, which measures the angle between two vectors and ignores their magnitude. It is the most common choice for text embeddings.
- Euclidean distance (L2), which measures the straight-line distance between two points in vector space. It is common in image retrieval.
- Dot product, which is similar to cosine similarity but also accounts for vector magnitude. It is often used when embeddings are already normalized.

Finding the exact nearest neighbor for a query vector by comparing it against every stored vector is called brute-force search. Brute-force search gives perfect accuracy but scales linearly with the number of stored vectors, which becomes too slow once a collection passes roughly one million vectors on typical hardware. Vector databases solve this by using approximate nearest neighbor (ANN) algorithms that trade a small amount of accuracy for large gains in query speed.

3. Core Architecture

Most vector databases share four core components:

- Ingestion pipeline: accepts raw vectors (and optional metadata) from the client, validates their dimensionality, and hands them to the indexing layer.
- Index: a data structure built on top of the stored vectors that makes approximate nearest neighbor search fast. The choice of index is the single biggest factor in a vector database's speed, memory usage, and accuracy.
- Metadata store: a secondary store, often a lightweight key-value or

document store, that holds structured fields attached to each vector (for example, a source URL, a timestamp, or a category tag) so that queries can be filtered before or after the similarity search runs.

- Query engine: accepts a query vector plus optional metadata filters, walks the index to find candidate matches, re-ranks them by exact distance, and returns the top-k results to the client.

4. Indexing Algorithms

4.1 HNSW (Hierarchical Navigable Small World)

HNSW builds a multi-layer graph where each vector is a node. The top layer contains a small number of nodes with long-range connections, and each layer below it becomes progressively denser, with the bottom layer containing every vector in the collection. A search starts at an entry point in the top layer and greedily moves toward the query vector, dropping down a layer each time no closer neighbor can be found at the current layer. HNSW typically achieves 95-99% recall (compared to brute-force search) while being 10-100x faster, which is why it is the default index in most modern vector databases, including Chroma, Qdrant, and Weaviate. Its main drawback is memory usage, since the full graph structure must be kept in RAM for fast traversal.

4.2 IVF (Inverted File Index)

IVF first clusters all stored vectors into a fixed number of groups, called "cells" or "Voronoi cells," using an algorithm such as k-means. At query time, the search only scans the vectors inside the cells whose centroids are closest to the query vector, rather than scanning every vector in the collection. IVF uses less memory than HNSW because it does not require an in-memory graph, but its accuracy depends heavily on the number of cells the search is allowed to probe: probing more cells improves recall but increases query time. IVF is commonly paired with product quantization (IVF-PQ) to compress vectors and further reduce memory usage, at some additional cost to accuracy.

4.3 Flat (Brute-Force) Index

A flat index stores vectors with no additional structure and compares the query vector against every stored vector. It guarantees 100% recall and is simple to implement, but query time grows linearly with collection size. Flat indexes are typically only used for small collections (under roughly 10,000 vectors) or as a ground-truth baseline when measuring the recall of an approximate index.

5. Metadata Filtering

Real-world queries are rarely "find the 10 most similar vectors" in isolation; they are usually "find the 10 most similar vectors where category = 'faq' and timestamp > 2024-01-01." Vector databases support this through two strategies:

- Pre-filtering: the metadata filter is applied first to narrow the candidate set, and the similarity search only runs over the surviving vectors. This is accurate but can be slow if the filter is very selective, since it may eliminate most of the index's optimizations.
- Post-filtering: the similarity search runs first over the full index to retrieve a larger candidate set (for example, the top 200 instead of the top 10), and the metadata filter is applied afterward to the candidates. This is faster in most cases but can return fewer than the requested number of results if the filter removes too many candidates from the initial set.

6. Popular Vector Database Implementations

- Chroma: an open-source, embedded vector database written primarily for local development and small-to-medium workloads. It runs in-process (no separate server required) and uses HNSW as its default index. Because it can run entirely in memory, it is popular for prototyping RAG pipelines.
- FAISS (Facebook AI Similarity Search): a library rather than a full database. It provides highly optimized implementations of Flat, IVF, HNSW, and product quantization indexes, and is frequently used as the underlying index engine inside other vector databases rather than being deployed directly by application developers.
- Pinecone: a fully managed, cloud-hosted vector database. It abstracts away index selection and infrastructure management, charging based on the number of vectors stored and queries per second.
- Qdrant: an open-source vector database written in Rust, designed to be run either embedded or as a standalone server, with built-in support for payload (metadata) filtering combined directly into the HNSW graph traversal rather than as a separate pre- or post-filtering step.
- Weaviate: an open-source vector database with a GraphQL query interface and built-in modules for generating embeddings automatically from raw text at ingestion time, rather than requiring the client to compute embeddings in advance.

7. Evaluating Retrieval Quality

Two metrics are most commonly used to evaluate a vector database's retrieval quality for a given configuration:

- Recall@k: the fraction of the true k nearest neighbors (as determined by brute-force search) that appear in the approximate index's top-k results. A recall@10 of 0.95 means that, on average, 9.5 out of the 10 true nearest neighbors were found.
- Latency (queries per second, or QPS): how many queries the index can serve per second at a given recall target. Recall and latency trade off against each other for every ANN algorithm: increasing the search parameters (for example, the number of IVF cells probed, or the HNSW search width) improves recall but reduces QPS.

8. Common Pitfalls

- Mismatched distance functions: an embedding model trained for cosine similarity will produce poor results if the vector database is configured to use Euclidean distance instead, since the two metrics can rank the same set of candidates differently.
- Re-embedding drift: if a collection's embeddings are generated by one model version and later queries are embedded with a newer or different model version, similarity scores become unreliable, because the two models do not share the same vector space.
- Under-provisioned memory for HNSW: because HNSW keeps its full graph in memory, a collection that grows past the available RAM will either fail to load or fall back to much slower disk-backed operation, depending on the database's implementation.
- Ignoring the recall/latency trade-off: teams sometimes tune an index purely for speed during development and are surprised when recall drops sharply once the collection grows to production scale, since the relationship between the two is rarely linear.

9. Summary

Vector databases exist to make approximate nearest neighbor search fast at scale, using indexing algorithms such as HNSW, IVF, and product quantization instead of brute-force comparison. The choice of index, distance function, and filtering strategy all directly affect the recall/latency trade-off a given application experiences, and mismatches between the embedding model used at ingestion time and the one used at query time are one of the most common sources of degraded retrieval quality in production systems.